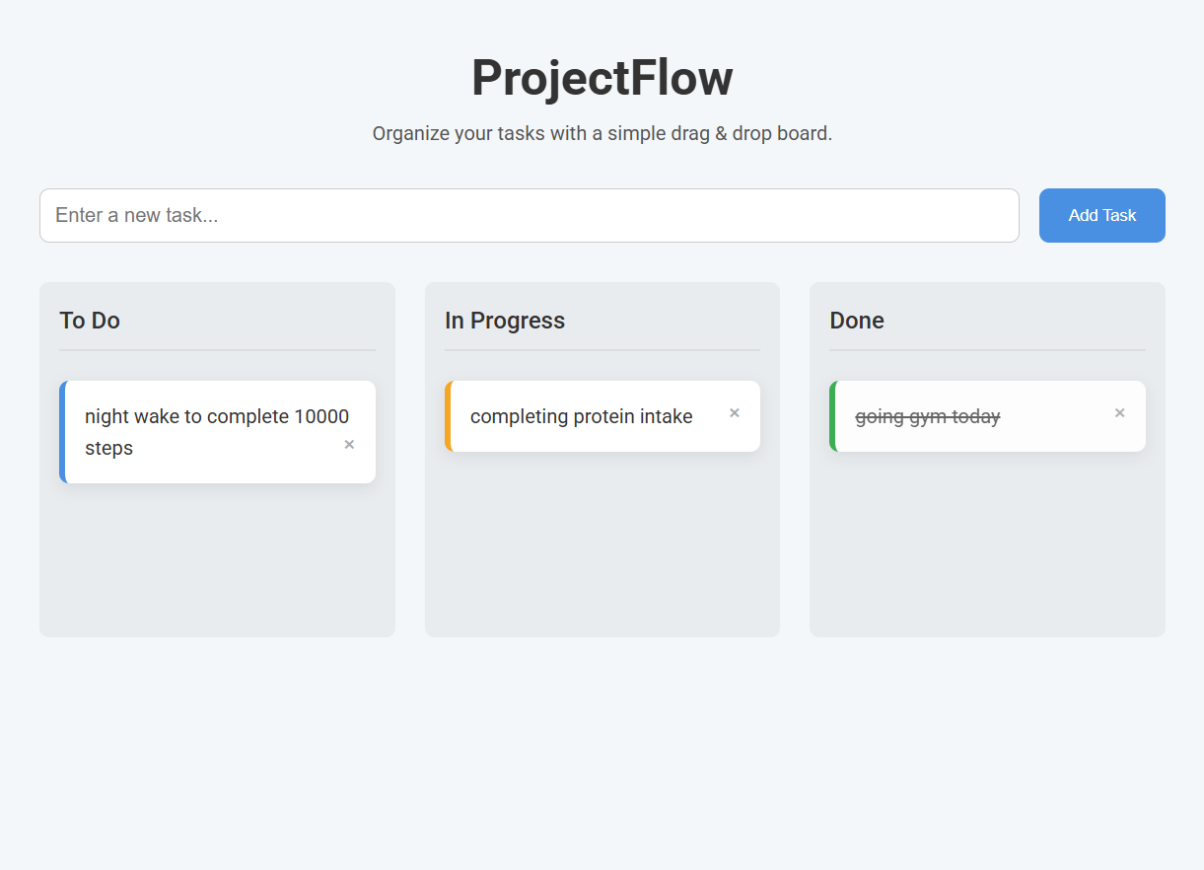
AN TODO-APP

Output:



HTML code:

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Kanban To-Do List</title>
  <link rel="stylesheet" href="style.css" />
  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
  <link
```

```
    href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;500;700&display=swap"
    rel="stylesheet"
   />
  </head>
  <body>
   <div class="app-container">
    <header class="app-header">
     <h1>ProjectFlow</h1>
     <p>Organize your tasks with a simple drag & drop board.</p>
    </header>

    <div class="task-input-container">
     <input type="text" id="task-input" placeholder="Enter a new task..." />
     <button id="add-task-btn">Add Task</button>
    </div>

    <main class="kanban-board">
     <div class="kanban-column" id="todo-column" data-column="todo">
      <h2 class="column-title">To Do</h2>
      <div class="tasks-container" data-column-body="todo"></div>
     </div>
     <div
      class="kanban-column"
      id="inprogress-column"
      data-column="inprogress"
     >
      <h2 class="column-title">In Progress</h2>
      <div class="tasks-container" data-column-body="inprogress"></div>
     </div>
```

```
      <div class="kanban-column" id="done-column" data-column="done">
        <h2 class="column-title">Done</h2>
        <div class="tasks-container" data-column-body="done"></div>
      </div>
    </main>
  </div>

  <script src="script.js"></script>
 </body>
</html>
```

CSS code:

```
:root {
 --bg-color: #f4f7f9;
 --column-bg: #e9ecef;
 --card-bg: #ffffff;
 --primary-color: #4a90e2;
 --text-dark: #333;
 --text-light: #555;
 --shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
 --border-radius: 8px;
 --todo-color: #4a90e2;
 --inprogress-color: #f5a623;
 --done-color: #28a745;
}

* {
 margin: 0;
 padding: 0;
```

```
  box-sizing: border-box;
}

body {
  font-family: "Roboto", sans-serif;
  background-color: var(--bg-color);
  color: var(--text-dark);
  line-height: 1.6;
}

.app-container {
  max-width: 1200px;
  margin: 2rem auto;
  padding: 0 2rem;
}

.app-header {
  text-align: center;
  margin-bottom: 2rem;
}

.app-header h1 {
  font-size: 2.5rem;
  font-weight: 700;
}

.app-header p {
  color: var(--text-light);
}
```

```
.task-input-container {
  display: flex;
  gap: 1rem;
  margin-bottom: 2rem;
}

#task-input {
  flex-grow: 1;
  padding: 0.75rem;
  border: 1px solid #ccc;
  border-radius: var(--border-radius);
  font-size: 1rem;
}

#add-task-btn {
  background-color: var(--primary-color);
  color: white;
  border: none;
  padding: 0.75rem 1.5rem;
  border-radius: var(--border-radius);
  cursor: pointer;
  font-weight: 500;
  transition: background-color 0.3s ease;
}

#add-task-btn:hover {
  background-color: #357abd;
}
```

```
.kanban-board {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1.5rem;
}

.kanban-column {
  background-color: var(--column-bg);
  padding: 1rem;
  border-radius: var(--border-radius);
}

.column-title {
  font-size: 1.2rem;
  font-weight: 500;
  margin-bottom: 1rem;
  padding-bottom: 0.5rem;
  border-bottom: 2px solid #ddd;
}

.tasks-container {
  min-height: 200px;
  padding: 0.5rem 0;
}

.task-card {
  background-color: var(--card-bg);
  padding: 1rem;
```

```
  border-radius: var(--border-radius);
  margin-bottom: 1rem;
  box-shadow: var(--shadow);
  cursor: grab;
  transition: box-shadow 0.3s ease, border-left-color 0.3s ease;
  border-left: 5px solid transparent;
}

.task-card:active {
  cursor: grabbing;
}

.task-card.dragging {
  opacity: 0.5;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.15);
}

.task-card .delete-btn {
  float: right;
  background: none;
  border: none;
  color: #aaa;
  cursor: pointer;
  font-size: 1.1rem;
  transition: color 0.3s ease;
}

.task-card .delete-btn:hover {
  color: #e74c3c;
```

```
}

/* State-specific colors */
#todo-column .task-card {
  border-left-color: var(--todo-color);
}

#inprogress-column .task-card {
  border-left-color: var(--inprogress-color);
}

#done-column .task-card {
  border-left-color: var(--done-color);
  opacity: 0.9;
}

#done-column .task-card span {
  text-decoration: line-through;
  color: var(--text-light);
}

@media (max-width: 768px) {
  .kanban-board {
    grid-template-columns: 1fr;
  }
  .app-container {
    padding: 0 1rem;
  }
}
```

JS Code:

```
document.addEventListener("DOMContentLoaded", () => {
 const taskInput = document.getElementById("task-input");
 const addTaskBtn = document.getElementById("add-task-btn");
 const columns = document.querySelectorAll(".tasks-container");

 let tasks = JSON.parse(localStorage.getItem("tasks")) || {
  todo: [],
  inprogress: [],
  done: [],
 };
 let draggedTask = null;

 const renderTasks = () => {
  columns.forEach((column) => {
   column.innerHTML = "";
   const columnId = column.dataset.columnBody;
   tasks[columnId].forEach((taskText, index) => {
    const taskCard = createTaskCard(taskText, columnId, index);
    column.appendChild(taskCard);
   });
  });
 };

 const createTaskCard = (text, columnId, index) => {
  const card = document.createElement("div");
  card.className = "task-card";
  card.draggable = true;
```

```
  card.dataset.id = index;
  card.dataset.column = columnId;

  const textSpan = document.createElement("span");
  textSpan.textContent = text;

  const deleteBtn = document.createElement("button");
  deleteBtn.className = "delete-btn";
  deleteBtn.innerHTML = "&times;";
  deleteBtn.addEventListener("click", (e) => {
   e.stopPropagation();
   deleteTask(columnId, index);
  });

  card.appendChild(textSpan);
  card.appendChild(deleteBtn);

  card.addEventListener("dragstart", handleDragStart);
  card.addEventListener("dragend", handleDragEnd);

  return card;
};

const saveTasks = () => {
  localStorage.setItem("tasks", JSON.stringify(tasks));
};

const addTask = () => {
  const taskText = taskInput.value.trim();
```

```
  if (taskText) {
    tasks.todo.push(taskText);
    taskInput.value = "";
    saveTasks();
    renderTasks();
  }
};

const deleteTask = (column, index) => {
  tasks[column].splice(index, 1);
  saveTasks();
  renderTasks();
};

const handleDragStart = (e) => {
  draggedTask = e.target;
  setTimeout(() => e.target.classList.add("dragging"), 0);
};

const handleDragEnd = (e) => {
  draggedTask.classList.remove("dragging");
  draggedTask = null;
};

columns.forEach((column) => {
  column.addEventListener("dragover", (e) => {
    e.preventDefault();
  });
```

```
  column.addEventListener("drop", (e) => {
    e.preventDefault();
    if (!draggedTask) return;

    const fromColumn = draggedTask.dataset.column;
    const fromIndex = parseInt(draggedTask.dataset.id);
    const toColumn = column.dataset.columnBody;

    const [task] = tasks[fromColumn].splice(fromIndex, 1);
    tasks[toColumn].push(task);

    saveTasks();
    renderTasks();
  });
 });

 addTaskBtn.addEventListener("click", addTask);
 taskInput.addEventListener("keypress", (e) => {
  if (e.key === "Enter") {
   addTask();
  }
 });

 renderTasks();
});
```